

Lab 5 - Мікросервіси з використанням Service Discovery та Config Server на базі Consul (10 балів)/на базі Kubernetes (15 балів)

Завдання базується на функціоналі, розробленому у попередньому завданні, і є його подальшим розвитком.

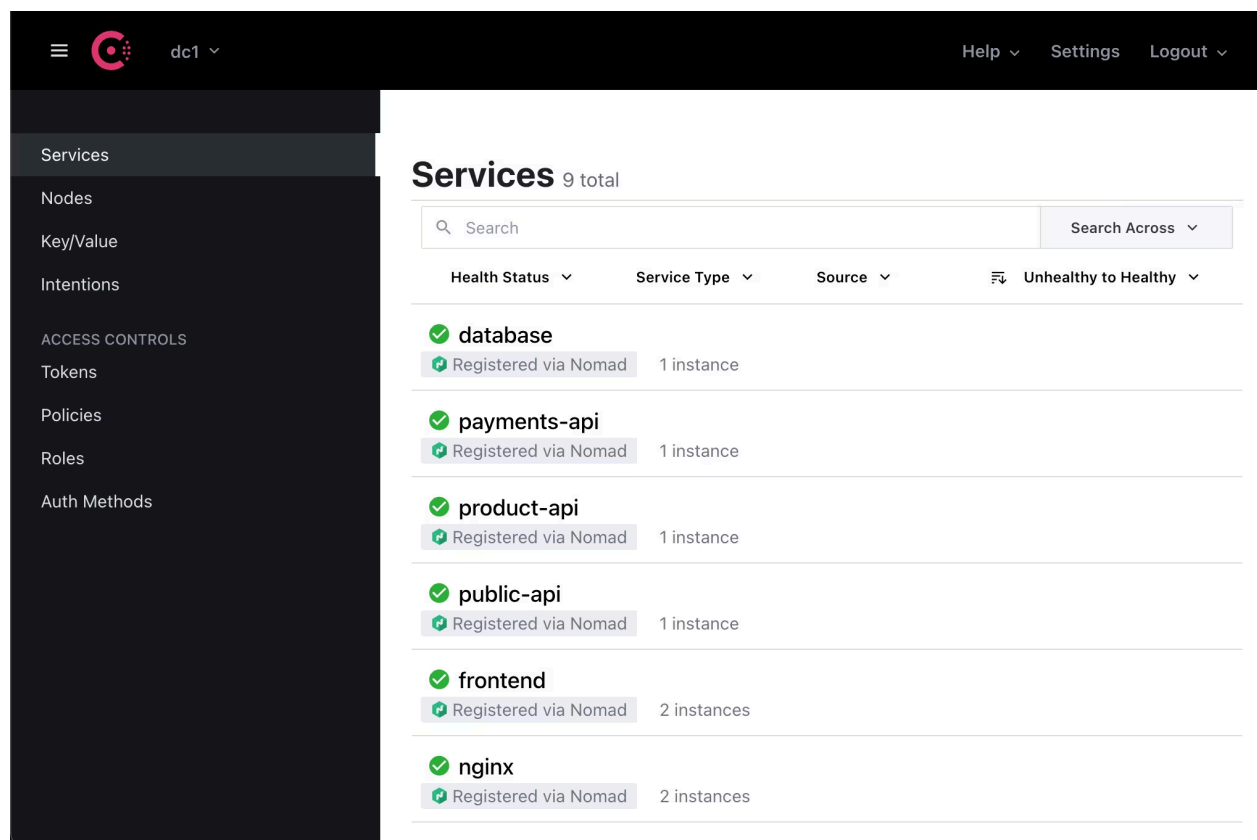
До системи необхідно додати *Consul* або *Kubernetes*, який буде виконувати роль *Service Register*, *Service Discovery* та *Config Server*.

Це дасть змогу явно не вказувати адреси мікросервісів та конфігурації для Hazelcast та Message Queue

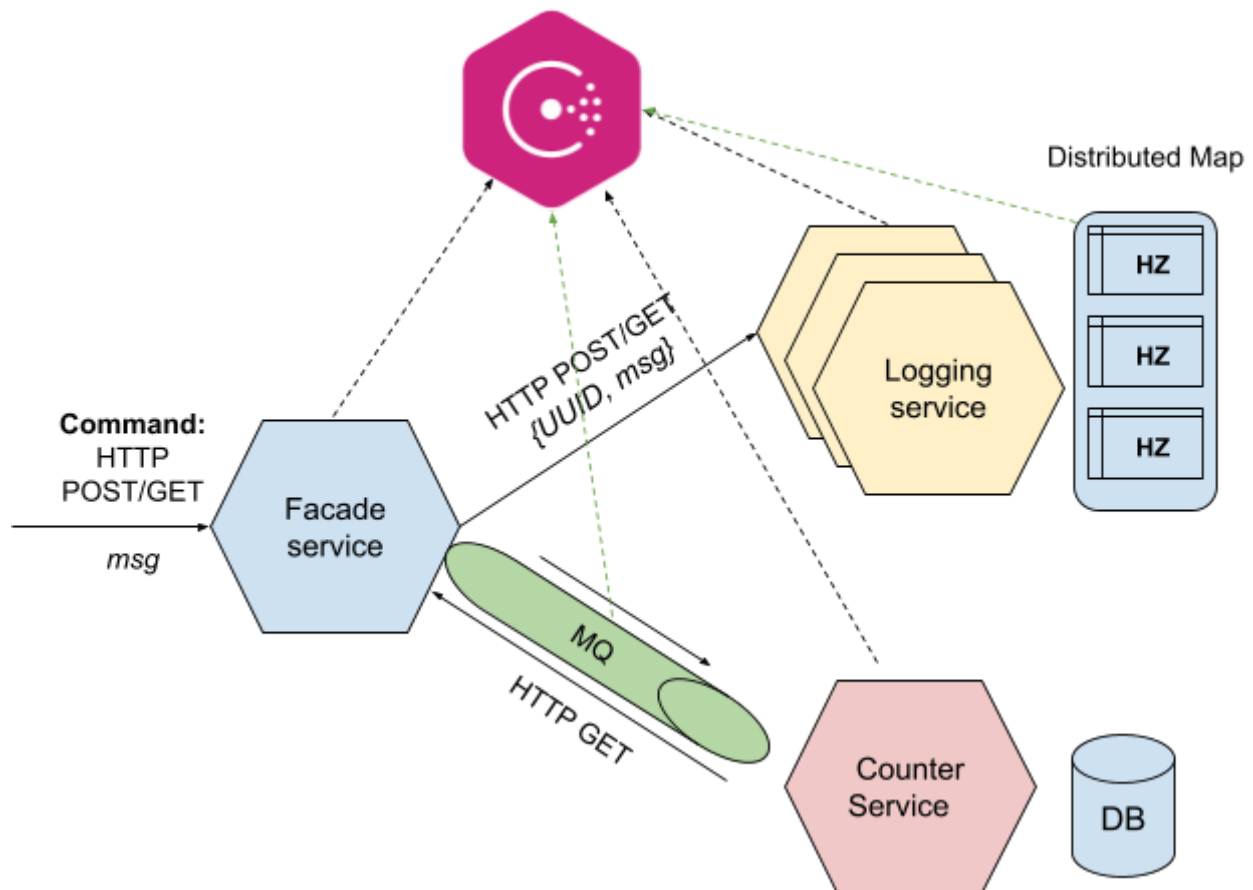
Також у випадку *Kubernetes*, всі сервіси будуть централізовано запускатись та мати можливість масштабуватись через *Kubernetes*

Вимоги:

- 1) Всі мікросервіси мають самостійно динамічно реєструватись при старті у *Consul/K8s*, кожного з сервісів може бути запущено кілька екземплярів (це буде відображатись як кількість instances на UI):
 - **facade-service**
 - **logging-service**
 - **counter-service**



- 2) При звертанні **facade-service** до **logging-service** та **counter-service**, IP-адреси (і порти) мають зчитуватися **facade-service** з *Consul/K8s*. Немає бути задано в коді чи конфігураціях статичних значень адрес.
- 3) Налаштування для клієнтів Hazelcast мають зберігатись як key/value у *Consul/K8s* і зчитуватись **logging-service**
- 4) Налаштування для Message Queue (адреса, назва черги, ...) мають зберігатись як key/value у *Consul/K8s* і зчитуватись **facade-service** та **counter-service**
- 5) Продемонструвати, що у випадку відключення екземпляру певного мікросервісу, це буде відображатися у *Consul/K8s* (відключений екземпляр сервісу змінить статус) , а виклики будуть перенаправлятися до інших працюючих екземплярів.



Примітка на випадок Kubernetes

У випадку використання Kubernetes, всі сервіси та супутні системи (Hazelcast, MQ, PostgreSQL) мають також запускатися у Kubernetes як окремі Pods.

Необхідно також буде надати у репозиторії всі конфігураційні yaml-файли для сервісів.

Як середовище для Kubernetes для локального запуску можна використовувати Minikube або аналогічне середовище.

Тестування продуктивності

Виконайте тестування продуктивності наведене у [першому завданні](#) (розділ Тестування продуктивності): два сценарії, та внесок кожного з мікросервісів в загальний час

Порівняйте отримані результати з результатами з першого та третього завдань, та наведіть їх у таблиці:

Test scenarios	Task 1 (in-mem)	Task 3 (DB)	Task 5 (final)
10 accounts	Total time:	Total time:	Total time:
	logging-service contribution:	logging-service contribution:	logging-service contribution:
	counter-service contribution:	counter-service contribution:	counter-service contribution:
1 account	Total time:	Total time:	Total time:
	logging-service contribution:	logging-service contribution:	logging-service contribution:
	counter-service contribution:	counter-service contribution:	counter-service contribution:

Вимоги до реалізації

Код всіх три мікросервісів має бути збережений у [GitHub](#) в одному репозиторії у гілці micro_consul

Вимоги до протоколу

У протоколі має міститись:

- посилання на GitHub з проектом

- скріншоти з POST/GET-запитами та відповіді на них
- вміст консолі кожного з мікросервісів якщо туди виводиться якась корисна інформація
- скріншот з Consul який би показував зареєстровані в ньому мікросервіси

Literature:

Consul in Docker: <https://learn.hashicorp.com/tutorials/consul/docker-container-agents>

(образи з новими версіями публікуються на

<https://hub.docker.com/r/hashicorp/consul>)

Consul Services: <https://www.consul.io/docs/discovery/services>

Register Service: <https://www.consul.io/api-docs/agent/service#register-service>

Consul Key-Value Config: <https://www.consul.io/docs/dynamic-app-config/kv>

Consul Key-Value: <https://www.consul.io/api-docs/kv>

Consul Client Libraries & SDKs:

<https://developer.hashicorp.com/consul/api-docs/libraries-and-sdks>

Залежно від обраної клієнтської бібліотеки, API буде відрізнятися, але ідея буде приблизно однакова.

Приклад

- Для реєстрації сервісу:

```
consul_service = consul.Consul()
```

```
consul_service.agent.service.register(...)
```

- Для отримання списку зареєстрованих сервісів по імені:

```
consul_service.health.service(service_name)
```